
SQ-DiT: SliderQuant for Diffusion Models

September 1, 2026

Simone Lidonnici - 2061343 Pietro Adamo Morelli - 2087359

Abstract

Diffusion Transformers (DiTs) have recently established a new baseline for image generation models, but their major limitations are the heavy computational and memory requirements. To mitigate these bottlenecks, Post-Training Quantization (PTQ) offers a practical approach to compress these models without retraining. In this project, we adapt SliderQuant, a PTQ algorithm originally designed for Large Language Models (LLMs), to the visual generation domain. Code is available at: <https://github.com/DoctorWho28/SQ-DiT>.

1. Introduction

Diffusion Transformers (DiT) set a new standard of generative AI, replacing the convolutional U-Net backbone of diffusion models with a Transformer architecture. DiTs rely on **Adaptive Layer Normalization (AdaLN)** to inject continuous timestep and discrete class conditionings into each block, dynamically adapting intermediate feature states at each denoising step. DiTs achieve remarkable results in high-fidelity image generation, but their large parameter count and iterative approach lead to prohibitive memory consumption (VRAM) and slow inference speeds. To mitigate these computational bottlenecks, Post-Training Quantization (PTQ) techniques compress the weights to lower-bit formats without requiring a full model retraining.

Recently, a PTQ framework named **SliderQuant** was introduced for Large Language Models (LLMs). SliderQuant exploits the fact that layers in deep networks have different sensitivities to quantization noise. It introduces a sliding-window distillation approach, optimizing quantization parameters over small groups of layers, and introduces learnable low-rank matrices (LoRA) to compensate for the quantization error.

Email:

Simone Lidonnici <lidonnici.2061343@studenti.uniroma1.it>,
Pietro Adamo Morelli <morelli.2087359@studenti.uniroma1.it>.

Deep Learning and Applied AI 2026, Sapienza University of Rome, 2nd semester a.y. 2025/2026.

SQ-DiT aims to adapt the core concepts of SliderQuant to Diffusion Transformers. The main difference is that SliderQuant calibrates LLMs over sequential tokens, whereas DiTs require optimizing across the temporal diffusion process. During the optimization process, the weights are quantized using fake quantization and only upon completion are the final weights packed into INT8 using our custom packing implementation to effectively save VRAM space. All experiments were conducted using the [facebook/DiT-XL-2-256](#) model, performing class-to-image generation at 256×256 resolution on the standard ImageNet-1k dataset.

2. Related Work

SliderQuant (Wang et al., 2026) serves as the foundation for SQ-DiT, from which we adapted the quantization and distillation concepts to the DiT architecture.

PTQ4DiT (Wu et al., 2024) and Q-DiT (Chen et al., 2024) serve as our primary foundational references for the state of the art in PTQ for DiTs.

3. Method

3.1. SliderQuant

SliderQuant builds on the premise that initial (shallow) and terminal (deep) layers have higher quantization sensitivity than intermediate ones. These boundary layers are optimized using a progressively expanding window, while the intermediate layers are optimized using a fixed size sliding window. In both cases, layers in the same window are quantized jointly.

To minimize quantization error, an optimizer is applied to a set of learnable parameters for each linear module, specifically a scaling factor (α) and two low-rank LoRA matrices (A and B).

To soften the impact on the weights and avoid excessive parameter perturbations, quantization is performed in two progressive stages: a partial quantization is applied according to a ratio γ , followed by the full quantization. We apply group-wise quantization to mitigate the impact of extreme weight values, confining outliers to local groups and pre-

serving the precision of the remaining parameters.

3.2. SQ-DiT implementation

Because the original SliderQuant framework cannot be easily transposed, we developed the entire quantization framework from scratch, implementing custom linear modules that internally encapsulate the learnable quantization parameters. Furthermore, to ensure numerical stability during the quantization process, the forward pass casts both the input activations and the weights, originally loaded in FP16, into FP32 before applying the scaling and fake quantization operations.

Timestep-Aware Window Distillation. We group transformer blocks into sliding windows and optimize them sequentially. For each window, we optimize the quantization parameters by minimizing the Mean Squared Error (MSE) between the output features of the quantized blocks and their original unquantized counterparts.

Unlike LLMs, where the input passes through each block only once, in DiT models the image is gradually denoised across multiple timesteps, passing through all the blocks at each step. Since data distributions change drastically throughout the denoising process, a quantization approach that propagates error across the timesteps would be unstable. For this reason, rather than allowing the error to propagate over time, each window jointly optimizes multiple timesteps. We provide the first block with the latents pre-calculated by the original FP16 model across all timesteps, while the subsequent blocks receive the output of the preceding quantized block.

By eliminating temporal propagation, we align the optimization with the SliderQuant paradigm used for LLMs, which don’t have noise accumulation between timesteps. This allows the LoRA matrices within each window to focus exclusively on compensating for spatial errors propagated by preceding quantized layers.

Mixed-Precision Architecture. Following the original observation that network layers exhibit varying sensitivities to quantization, we decided to augment the SliderQuant approach by applying a mixed-precision policy. Deep and shallow transformer blocks are highly sensitive and are therefore quantized to higher bit-widths while intermediate layers, which are more robust to quantization noise, can be quantized to lower bit-widths.

Additionally, crucial modules responsible for condition injection require special treatment. The initial **Embedding** module, which maps discrete time and class conditions into a continuous embedding space, is kept in FP16. Since this conditioning guides the entire generative process, any error introduced here would cascade through all subsequent

layers and degrade the feature representations. Similarly, the **AdaLN** module, responsible for injecting temporal and class features into the network, is quantized to higher precision. Aggressively quantizing this layer would cause the network to lose temporal awareness and class concepts, leading to generation failures.

Using this mixed-precision approach has a slight impact on VRAM utilization, but this is negligible compared to the benefits on generation stability.

Weights Packing. To effectively have an impact on the model size and VRAM utilization, we implemented a custom `WXAXLinear` module as a replacement for standard linear modules. This module physically packs low-bit quantized weights into INT8 using bitwise shift and masking operations.

When loading the model to generate an image, we replace the original linear modules with their `WXAXLinear` counterparts and load the packed weights into memory. During inference, the layer performs on the fly unpacking, restoring the weights to FP16 dynamically to continue the image generation process.

4. Evaluation

Sensitivity Analysis Following the principle introduced by SliderQuant, we conducted a sensitivity analysis to determine the optimal configuration of high-precision boundary layers for DiTs. Our sensitivity evaluation methodology is inspired by the *Sensitivity-Aware Mixed Precision Quantization* (Huang & Badaoui, 2025) for LLMs, which estimates layer sensitivity by quantizing a single layer at a time, while keeping the rest of the network in FP16. In the original approach, divergence is measured using the Jensen-Shannon Divergence (JSD) between output probability distributions. However, since DiTs output continuous values rather than probabilities, JSD is inapplicable. We replaced it with the linear inverse of the *Signal-to-Noise Ratio* (SNR) (Roux et al., 2018) to evaluate the continuous error.

Optimization Parameters The training was conducted using a *GPU Nvidia RTX 4090* and based on its computational capabilities, we selected the hyperparameters to balance the computational time required and the quantization quality. To fully exploit the GPU computing power and accelerate the distillation process, we implemented a batch-stacking mechanism where inputs across all timesteps are concatenated along the batch dimension. This allows the model to compute the forward pass and the MSE loss for all timesteps simultaneously, within a single GPU operation.

Optimization was conducted for **30 epochs** per quantiza-

tion stage (60 in total), which proved sufficient for the learnable parameters to converge and effectively compensate for the quantization noise, avoiding overfitting. To ensure that the calibrated parameters generalize well across different image concepts, we uniformly sampled a subset of **20 classes**, sampling one every 50 classes. Furthermore, we optimized the model over **20 inference steps**, which we found to be sufficient to generate images of acceptable visual quality.

Scaling up the calibration process by including more classes or inference timesteps would have been prohibitively expensive in terms of computational time and hardware costs.

Quantitative evaluation To evaluate the generative quality of our quantized models against the original, we used several standard quantitative metrics, specifically **Fréchet Inception Distance (FID)** (Heusel et al., 2018), **spatial FID (sFID)** (Nash et al., 2021) and **Inception Score (IS)** (Barratt & Sharma, 2018).

Additionally, we utilized **CMMD** (CLIP Maximum Mean Discrepancy) (Jayasumana et al., 2024), a modern and highly sample-efficient metric, which provides a robust assessment of image quality.

Table 1 presents the evaluation of our DiT models with W8A8 (8-bit weights and activations) and W4A8 (4-bit weights and 8-bit activations) configurations. In both configurations, the higher-precision layers in our mixed-precision policy are in 8-bit. The **model size** is reported to quantify the benefits in VRAM utilization achieved by our INT8 packing mechanism.

Table 1. Evaluation comparison

	Size	FID ↓	sFID ↓	IS ↑	CMMD ↓
FP16	1.58 GB	24.72	59.3	318 ± 7.9	0.236
W8A8	0.97 GB	24.48	57.8	322 ± 6.0	0.239
W4A8	0.78 GB	23.49	55.7	314 ± 6.7	0.236

Our quantization framework achieves significant memory reductions while fully preserving generative quality. Moving from the FP16 baseline to W8A8 reduces the model size by more than a third and the W4A8 configuration effectively halves it. We observe a numerical improvement in the FID and sFID scores for the quantized models, which is a known artifact of how these metrics measure distribution distances, as they can sometimes reward the slight smoothing effect introduced by quantization. For a more reliable assessment of generative fidelity, we look at the CMMD metric, which remains stable across all models, demonstrating that quantization preserves the visual integrity, as shown in Figure 1.



Figure 1. Visual comparison of generated samples across two different classes

5. Conclusion

In this project, we introduced SQ-DiT, an adaptation of the SliderQuant post-training quantization framework, specifically designed for Diffusion Transformers. By addressing the temporal and architectural characteristics of DiTs, we successfully implemented a timestep-aware window distillation approach combined with a mixed-precision quantization policy. Furthermore, our custom weight-packing module ensures tangible reductions in model size and VRAM utilization. The results highlight the effectiveness of our framework in making diffusion models more computationally efficient without compromising visual quality.

Future work could include comparing SQ-DiT with other quantization methodologies, with a wider range of quantization configurations, as well as applying the approach to higher-resolution diffusion models (for example, 512×512).

Statement on the use of AI

AI was used for literature exploration, mainly to understand the state of the art related to diffusion transformers.

For the code, AI was used to identify the most suitable libraries and functions to use, as well as to implement scripts to support the project (such as image generation and metric calculation). In these cases, AI provided the basic structure of the code, which we then adapted to work for our framework. It also assisted us in analyzing and resolving technical issues, such as optimizing the batch-stacking mechanism to make the most of the RTX 4090 GPU.

In the report, we used it to rephrase sentences and create formulas and tables using existing data.

References

- Barratt, S. and Sharma, R. A note on the inception score, 2018. URL <https://arxiv.org/abs/1801.01973>.
- Chen, L., Meng, Y., Tang, C., Ma, X., Jiang, J., Wang, X., Wang, Z., and Zhu, W. Q-dit: Accurate post-training quantization for diffusion transformers, 2024. URL <https://arxiv.org/abs/2406.17343>.
- Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., and Hochreiter, S. Gans trained by a two time-scale update rule converge to a local nash equilibrium, 2018. URL <https://arxiv.org/abs/1706.08500>.
- Huang, J. and Badaoui, A. Sensitivity aware mixed precision quantization v1, 2025. URL <https://huggingface.co/blog/badaoui/sensitivity-aware-mixed-precision-quantizer-v1>.
- Jayasumana, S., Ramalingam, S., Veit, A., Glasner, D., Chakrabarti, A., and Kumar, S. Rethinking fid: Towards a better evaluation metric for image generation, 2024. URL <https://arxiv.org/abs/2401.09603>.
- Nash, C., Menick, J., Dieleman, S., and Battaglia, P. W. Generating images with sparse representations, 2021. URL <https://arxiv.org/abs/2103.03841>.
- Roux, J. L., Wisdom, S., Erdogan, H., and Hershey, J. R. Sdr - half-baked or well done?, 2018. URL <https://arxiv.org/abs/1811.02508>.
- Wang, S., Li, C., Kang, Y., Fan, J., Ou, Z., and Yao, A. Slid-erquant: Accurate post-training quantization for llms, 2026. URL <https://arxiv.org/abs/2603.25284>.
- Wu, J., Wang, H., Shang, Y., Shah, M., and Yan, Y. Ptq4dit: Post-training quantization for diffusion trans-formers, 2024. URL <https://arxiv.org/abs/2405.16005>.

Appendix

Final weights formula After the optimization, the final quantized weights $\tilde{W}$ are calculated using the learnable parameters as follows:

$$\tilde{W} = W \cdot \alpha + A \times B$$

Sensitivity analysis formulas The linear inverse of the SNR metric is detailed by the formula:

$$\text{NSR} = 10^{\left(\frac{-\text{SNR}}{10}\right)}$$

The sensitivity score of each layer l is then defined by:

$$\text{Score}_l = \text{NSR}_l \cdot \left(1 + \lambda \cdot \frac{\text{magnitude}_l}{\max_k \text{magnitude}_k}\right)$$

In our case, we set $\lambda = 0.1$.

Quantization hyperparameters The hyperparameters used in our experiments are listed below:

Table 2. Hyperparameters

Configuration	Setting
Epoch	30
Classes	20
Inference timesteps	20
Batch size	4
Layer shallow	3
Layer deep	2
Window size	2
Window step	1
γ	0.5
Rank of LoRA	16
Quantization group size	128

The parameters are equal for both W8A8 and W4A8 configurations.